GIFT School of Engineering
and Applied Sciences

Spring 2026

CS-367 Object Oriented Analysis and Design – Lab

Lab-1 Manual

Introduction OOP Concepts

Exercise 1: Understanding Classes and Objects

Task:

Create a Java class Car with data members: brand, model, year and price.
Add methods to:

- Display car details.
- Update the price.

Sample Output:

```
Car Brand: Toyota
Model: Corolla
Year: 2021
Price: 4500000
```

Exercise 2: Encapsulation and Data Hiding

Task:

Modify the Car class to make all data members private. Provide getter and setter methods.
Demonstrate how encapsulation ensures data protection.

Hint:

Use private and public keywords appropriately.

Exercise 3: Inheritance

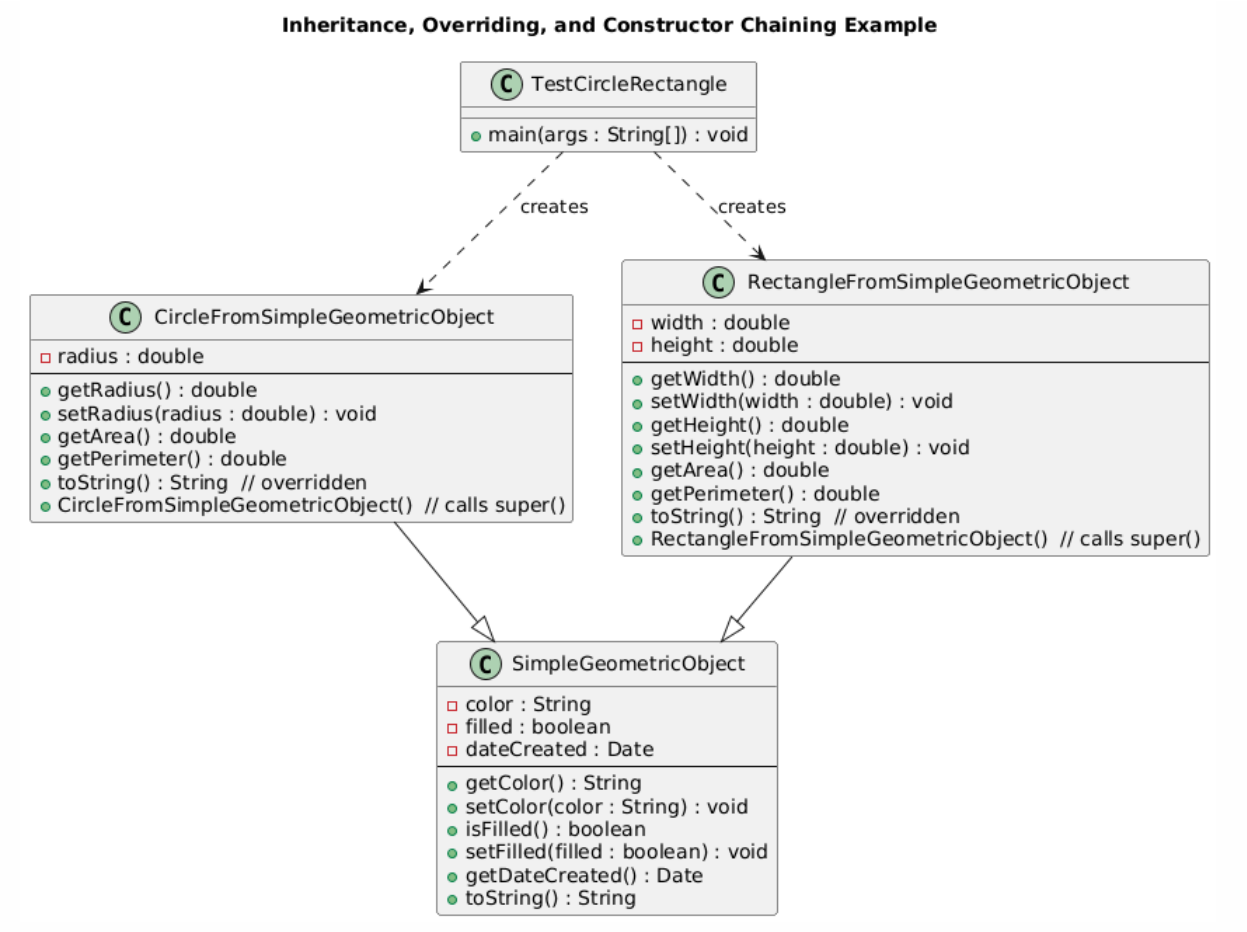
Objective

To implement the concepts of **Inheritance**, **Method Overriding**, and **Constructor Chaining** in Java.

Scope

Inheritance allows specialized classes (subclasses) to reuse and extend the attributes and methods of a general class (superclass).
Through **constructor chaining**, constructors from the superclass are invoked to initialize inherited data.
Method overriding enables subclasses to redefine specific methods to provide specialized behavior.

Example



Task:

Implement the class diagram given above in java.

Exercise 3: Polymorphism

Objective

To implement the concept of Polymorphism

Scope

Polymorphism means "many forms." It allows the same method to behave differently depending on which object it belongs to.

Example 1: Compile-Time Polymorphism (Method Overloading)

Task:

Create a class named Calculator.

Define multiple add() methods that:

- Accept different **types** of arguments (e.g., int, double).
- Return the sum of the given arguments.

In the main() method:

- Create an object of the Calculator class.
- Call each version of the add() method and print the result.

Example 2: Runtime Polymorphism (Method Overriding)

Consider the following class

```
public class father {
    int age;
    father(int x) {
        age=x;
    }
    public void Iam() {
        System.out.println("I AM FATHER and My AGE = %d", age);
    }
}
```

Task:

Derive two sub classes son and daughter from the above class and override the method Iam() in each of the subclass and prints an appropriate message in each overridden method. You should also define a suitable constructor for each of these sub classes. Now write a test class to check the functionality of above example. In main() method create three objects one for each father, son and daughter respectively and call their Iam() method non polymorphically. Then create an array of father references of size ten and initialize this array randomly with the objects of daughter and son. Then traverse this array and call the Iam() method polymorphically and display the results.

Exercise 4: Abstraction using Abstract Class or Interface

Objective

To implement the concept of Abstraction in Java using Abstract Classes and Interfaces.

Scope

Abstraction in Java is achieved using abstract classes and interfaces. An interface defines a contract that classes must follow—it contains only abstract methods (and optionally default or static ones) without implementation. A class implements an interface to provide concrete behavior.
An abstract class can contain both abstract and concrete methods. It cannot be instantiated, but its subclasses extend it to implement the abstract methods and reuse common code.

Task 1:

- Create an abstract class named Restaurant.
- Add a method to print the restaurant's name.
- Add the following abstract methods:
  - totalPrice(double basePrice)
  - menuItems()
  - location()
- Create a class called McDonalds that extends Restaurant.
- Implement all abstract methods in the McDonalds class.
- Add logic in totalPrice() to calculate the total meal price including a 6% tax.
- Add a boolean method hasPlayPlace() returning true if the restaurant has a play place.
- Create a constructor that sets the name, location, and play place availability.

Task 2:

- Create an Interface named Restaurant
- Declare the following **abstract methods**:
  - double totalPrice(double basePrice)
  - void menuItems()
  - void location()
- Create a class called McDonalds that implements Restaurant.
- Implement all the methods defined in the interface.
- Add logic in totalPrice() to calculate the meal price including a 6% tax.
- Add a method hasPlayPlace() returning true or false.
- Add a constructor that sets the restaurant's name, location and play place availability.

Exercise 5: MS Visio and StarUML

Task:

Download and complete the installation before the next Lab.

MS Visio

StarUML Download Link

StarUML Documentation Link